

E1 — Procesamiento de datos con pandas

Explicación de la práctica y de los conceptos teóricos para repasar de cara al examen.

1. Qué se ha hecho en la práctica

Objetivo: cargar un CSV de ventas con datos "sucios", detectar las filas incorrectas, quedarse con las buenas, calcular resúmenes y exportar los resultados.

Dataset: Datos/ventas.csv tiene 150 filas y 6 columnas: fecha, region, producto, unidades, precio_unitario y cliente_id.

Parte	Qué se hace	Herramientas clave
1. Diagnóstico	Cargar el CSV y revisar tamaño, tipos y vacíos	pathlib.Path, read_csv, shape, dtypes, isna().sum()
2. Validación	Separar filas válidas e inválidas y calcular el importe	pd.to_numeric(errors="coerce"), máscaras booleanas, ~
3. Agregaciones	Importe por región, top 3 productos y clientes que repiten	groupby, sum, sort_values, head, value_counts
4. Exportación	Guardar el CSV limpio y un JSON con métricas de calidad	to_csv(index=False), json.dump, with open

Resultados obtenidos:

- 140 filas válidas y 10 inválidas (coincide con el checkpoint).
- Importe total: 96 598,17.
- Región con más importe: Este. Producto con más importe: Gateway.
- Archivos generados: Datos/ventas_limpias.csv y Datos/calidad_datos.json.

Problemas que tenía el CSV:

- unidades se leía como texto porque contenía el valor "na".
- Había valores vacíos (2 en unidades y 1 en precio_unitario).
- Había valores iguales a 0 o negativos en unidades y en precio_unitario.

2. Conceptos teóricos

2.1 Rutas con `pathlib`

```
from pathlib import Path
carpeta_script = Path(__file__).parent          # carpeta donde está el script
carpeta_datos = carpeta_script.parent / "Datos" # se sube un nivel y se entra en
Datos
```

- `__file__` es la ruta del script que se está ejecutando.
- `.parent` devuelve la carpeta que contiene esa ruta. Encadenarlo sube un nivel cada vez.
- El operador `/` une trozos de ruta y funciona igual en Windows, Linux y Mac.
- **Por qué no usar rutas absolutas** (`C:/Users/...`): solo funcionan en tu ordenador.
- **Por qué no usar rutas relativas simples** (`Path("Datos")`): dependen de la carpeta desde la que ejecutes el comando (el *directorio de trabajo*), no de dónde esté el script.

2.2 DataFrame y diagnóstico inicial

- **DataFrame**: tabla de pandas con filas (identificadas por un *índice*) y columnas con nombre.
- **Series**: una sola columna de un DataFrame.
- `df.shape` devuelve una tupla (filas, columnas).
- `df.dtypes` devuelve el tipo de cada columna: `int64` y `float64` son numéricos; `str` (en versiones antiguas de pandas, `object`) es texto.
- `df.isna().sum()` cuenta los vacíos por columna. `isna()` devuelve True/False en cada celda, y `sum()` cuenta los True, porque True vale 1.
- `df.head(n)` muestra las primeras n filas.

Idea clave: basta **un solo valor de texto** en una columna numérica para que pandas lea la columna entera como texto. Revisar los `dtypes` es la forma más rápida de detectarlo.

2.3 Valores nulos: `NaN`

- `NaN` ("Not a Number") representa un dato ausente.
- Cualquier comparación con `NaN` da `False` (`NaN > 0`, `NaN == 0`, incluso `NaN == NaN`).
- Por eso, para detectar nulos se usa `isna()` y nunca `== NaN`.

2.4 Conversión de tipos: `pd.to_numeric`

```
pd.to_numeric(serie, errors="coerce")
```

errors=	Qué hace con un valor no convertible
"raise" (por defecto)	Lanza un error y el programa se para
"coerce"	Lo convierte en NaN

Con "coerce", el texto basura pasa a ser NaN y la regla `> 0` lo descarta automáticamente.

2.5 Máscaras booleanas (filtrado)

```
filas_ok = (df["unidades"] > 0) & (df["precio_unitario"] > 0)
validos = df[filas_ok]
errores = df[~filas_ok]
```

- Una **máscara** es una Series de True/False, con un valor por fila.
- `df[mascara]` se queda solo con las filas que valen True.
- Operadores lógicos en pandas:

Operador	Significado	Equivale en Python normal a
&	Y	and
	O	or
~	NO (invierte)	not

- **Hay que poner paréntesis** alrededor de cada condición, porque `&` tiene más prioridad que `>`.
- **No se puede usar** `and` / `or` con Series: dan el error *"The truth value of a Series is ambiguous"*.

2.6 .copy()

- `df[mascara]` puede devolver una "vista" ligada al DataFrame original.
- Si después se añade una columna, pandas avisa con `SettingWithCopyWarning`.
- `.copy()` crea un DataFrame independiente: se puede modificar sin afectar al original y sin avisos.

2.7 Columnas derivadas

```
validos["importe"] = validos["unidades"] * validos["precio_unitario"]
```

- Las operaciones entre columnas son **vectorizadas**: se aplican a todas las filas a la vez, sin bucles `for`, y son mucho más rápidas.
- Se calcula solo en las filas válidas para no generar importes absurdos (negativos o NaN).

2.8 Agregaciones con `groupby`

```
validos.groupby("region")["importe"].sum().sort_values(ascending=False)
```

Sigue el patrón **split - apply - combine**:

1. **Split**: separa las filas en grupos (uno por región).
2. **Apply**: aplica una función a cada grupo (`sum` , `mean` , `count` , `max` ...).
3. **Combine**: junta los resultados en una Series con una fila por grupo.

Otras funciones útiles:

- `sort_values(ascending=False)` ordena de mayor a menor.
- `head(3)` devuelve los 3 primeros. `nlargest(3)` hace lo mismo en un solo paso.
- `value_counts()` cuenta cuántas veces aparece cada valor, ordenado de más a menos.
- `serie[serie > 1]` filtra una Series con una máscara, igual que se hace con un DataFrame.

2.9 Exportación

CSV:

```
validos.to_csv(ruta, index=False)
```

`index=False` evita guardar el índice como una columna extra sin nombre.

JSON:

```
with open(ruta, "w", encoding="utf-8") as f:  
    json.dump(diccionario, f, indent=4)
```

- **JSON** es un formato de texto con estructura clave-valor. Es el equivalente a un diccionario de Python y el estándar para intercambiar datos entre programas y APIs.
- `with open(...)` abre el archivo y lo **cierra automáticamente** al terminar el bloque, aunque haya un error. Esto se llama *context manager*.
- Modos de apertura: `"r"` leer, `"w"` escribir (sobrescribe el contenido), `"a"` añadir al final.
- `indent=4` indenta el JSON para que sea legible.
- `json.dump` escribe en un archivo, mientras que `json.dumps` devuelve un string.
- El JSON solo acepta tipos básicos de Python (`int` , `float` , `str` , `list` , `dict` , `bool` , `None`). Por eso se convierte el resultado de pandas con `float(...)`.

2.10 Funciones: type hints y docstrings

```
def validar_ventas(frame: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:  
    """Explica qué hace la función, qué recibe y qué devuelve."""
```

- **Type hints** (: `pd.DataFrame` , -> `tuple[...]`) indican el tipo esperado de los argumentos y del resultado. Python **no los comprueba** al ejecutar: sirven de documentación y para que el editor ayude.
- **Docstring**: el texto entre `"""` justo debajo del `def` . Es la documentación oficial de la función y se consulta con `help(validar_ventas)` .
- Una función que devuelve dos cosas devuelve una **tupla**, que se "desempaqueta" así:
`validos, errores = validar_ventas(ventas)` .

2.11 Calidad de datos (concepto general)

- **Validar** es comprobar que cada dato cumple unas reglas de negocio (aquí: cantidades y precios positivos).
- Las filas inválidas **no se borran sin más**: se guardan aparte (`errores`) para poder revisarlas.
- Un **informe de calidad** (filas totales, válidas, inválidas) permite comprobar de un vistazo si los datos de entrada han empeorado.
- Pipeline típico: **cargar** → **validar** → **transformar/agregar** → **exportar**.

2.12 Git: guardar y subir el trabajo

Comando	Qué hace
<code>git status</code>	Muestra qué archivos han cambiado y cuáles están preparados
<code>git add <ruta></code> / <code>git add .</code>	Prepara los cambios (<i>staging area</i>)
<code>git commit -m "mensaje"</code>	Guarda una "foto" de los cambios preparados en el historial local
<code>git push</code>	Sube los commits al repositorio remoto (GitHub)

- **Untracked**: archivo nuevo que git todavía no controla.
- `git add` no muestra ningún mensaje cuando funciona bien.

3. Preguntas típicas de examen (E1)

1. **¿Por qué una columna numérica puede aparecer como texto?** Porque contiene al menos un valor no numérico (`"na"` , `"abc"` ...), y pandas lee entonces la columna entera como texto.
2. **¿Qué hace `errors="coerce"` ?** Convierte en `NaN` los valores que no se pueden pasar a número, en lugar de lanzar un error.
3. **¿Por qué los `NaN` quedan fuera con la regla `> 0` ?** Porque cualquier comparación con `NaN` devuelve `False` .

4. **¿Por qué se usa `&` y no `and` al filtrar?** Porque `and` no funciona con Series (no puede decidir un único True/False). `&` compara fila a fila.
5. **¿Para qué sirve `~`?** Para invertir una máscara: pasar de filas válidas a inválidas.
6. **¿Qué significa `split-apply-combine`?** Es el funcionamiento de `groupby` : separar en grupos, aplicar una función a cada uno y juntar los resultados.
7. **¿Para qué sirve `index=False` en `to_csv`?** Para no guardar el índice como una columna extra.
8. **¿Qué ventaja tiene `with open(...)`?** Que cierra el archivo automáticamente.
9. **¿Por qué construir la ruta con `Path(__file__)`?** Para que funcione desde cualquier carpeta y en cualquier ordenador.
10. **¿Se comprueban los `type hints` al ejecutar?** No, son solo documentación.